

Test Driven Development en Java

Jour 2 — Mocks, Stubs & Techniques de Tests

Formation professionnelle · 3 jours · Niveau intermédiaire

Programme — Jour 2

01 Rappels Jour 1 & introduction

03 Stub, Fake, Spy, Mock — Différences

05 Bibliothèques de mocking

07 Mockito — Fonctionnalités avancées

09 Qualité du code de test

11 Styles de TDD

02 Les Doubles de Test — Théorie

04 Implémentation sans bibliothèque

06 Mockito — Fondamentaux

08 Fixtures et organisation des tests

10 Tests basés sur la responsabilité

12 Travaux pratiques

| 01 · Rappels & Introduction Jour 2

Où en sommes-nous ?



Rappels du Jour 1

- Cycle **Red** → **Green** → **Refactor**
- Le patron **AAA** (Arrange, Act, Assert)
- Tests **FIRST** (Fast, Independent, Repeatable...)
- **JUnit 5** : annotations, assertions, tests paramétrés
- **@Nested** pour l'organisation des tests
- Tests des **exceptions** avec `assertThrows`
- **Refactoring** sécurisé grâce aux tests
- **Conception émergente** et principes SOLID
- **Repository Pattern** pour la testabilité
- **AssertJ** pour les assertions fluides

Objectifs du Jour 2

- Comprendre la **théorie des doubles de test**
- Distinguer **Mock, Stub, Fake, Spy, Dummy**
- Implémenter des doubles **manuellement** (sans lib)
- Maîtriser **Mockito** pour le mocking en Java
- Utiliser `when()`, `verify()`, `ArgumentCaptor`
- Écrire des tests de **haute qualité**
- Comprendre les **styles de TDD** (classique vs London)

02 · Les Doubles de Test — Théorie

Isoler le code sous test

? Pourquoi des doubles de test ?

Le problème des dépendances

```
public class VirementService {  
    private final BanqueAPI banqueAPI; // ← service externe réel  
    private final EmailService emailService; // ← envoie de vrais emails  
    private final UserRepository repository; // ← vraie base de données  
  
    public void effectuerVirement(Virement v) {  
        // Comment tester ça ?  
        banqueAPI.transférer(v); // appel réseau réel  
        emailService.notifier(v.getDest()); // vrai email envoyé !  
        repository.save(v); // vraie insertion BDD !  
    }  
}
```

La solution : les Doubles de Test

*Un **Test Double** est un objet qui remplace une dépendance réelle dans les tests, permettant d'isoler le code sous test.*

— Gerard Meszaros, xUnit Test Patterns

Pourquoi ?

- Isoler le **code sous test** de ses dépendances
- Contrôler le **comportement** des dépendances
- Rendre les tests **rapides** (pas d'appels réseau/BDD)
- Tester des **scénarios difficiles** (erreurs réseau, timeouts)

Les 5 types de Doubles de Test

Dummy

Objet passé mais jamais utilisé. Remplit un paramètre obligatoire.

Stub

Retourne des réponses prédéfinies. Ne vérifie rien.

Spy

Enregistre les appels reçus. Peut retourner des valeurs réelles ou stubées.

Mock

Vérifie que certaines interactions ont bien eu lieu. Orienté comportement.



Dummy — Objet fantoche

Définition

Objet passé en paramètre mais **jamais utilisé** dans le test. Sert uniquement à satisfaire la signature d'une méthode ou d'un constructeur.

Exemple

```
@Test
void creerCommande_ArticlesValides_ReussitCreation() {
    // Logger est requis par le constructeur mais inutile pour ce test
    Logger dummyLogger = new Logger() {
        public void log(String msg) {} // implémentation vide
    };

    CommandeService service = new CommandeService(repo, dummyLogger);
    Commande commande = service.creer(articles);

    assertNotNull(commande);
}
```

Stub — Réponse prédéfinie

Définition

Retourne des **valeurs prédéfinies** en réponse aux appels. Ne vérifie pas comment il est utilisé. Sert à fournir un **état indirect** au test.

Exemple

```
// Stub manuel
class StubMeteoService implements MeteoService {
    private final String meteoRetournee;
    public StubMeteoService(String meteo) { this.meteoRetournee = meteo; }

    @Override
    public String getMeteo(String ville) {
        return meteoRetournee; // toujours la même réponse
    }
}

// Utilisation dans le test
MeteoService stub = new StubMeteoService("Ensoleillé");
```

Spy — Espion

Définition

Enregistre les **appels reçus** (méthodes appelées, arguments passés). Peut déléguer à l'implémentation réelle ou retourner des valeurs stubées. Permet de vérifier **comment** une dépendance a été utilisée.

Exemple manuel

```
class SpyEmailService implements EmailService {
    private List<String> emailsEnvoyes = new ArrayList<>();

    @Override
    public void envoyer(String destinataire, String message) {
        emailsEnvoyes.add(destinataire); // enregistre l'appel
        // pas de vrai envoi !
    }

    public List<String> getEmailsEnvoyes() { return emailsEnvoyes; }
}
```

Mock — Vérificateur de comportement

Définition

Vérifie que certaines **interactions ont bien eu lieu** : quelles méthodes ont été appelées, avec quels arguments, combien de fois. Orienté **comportement** plutôt qu'état.

Différence clé

- **Stub** → vous contrôlez l'**entrée** (ce que la dépendance retourne)
- **Mock** → vous vérifiez la **sortie** (ce que le code sous test *fait* avec la dépendance)

Fake — Fausse implémentation

Définition

Implémentation **fonctionnelle mais simplifiée** qui n'est pas adaptée à la production. Plus réaliste qu'un stub, mais plus simple que la vraie implémentation.

Exemples classiques

```
// Fake Repository – implémentation en mémoire
class InMemoryUserRepository implements UserRepository {
    private Map<Long, User> store = new HashMap<>();

    @Override
    public User save(User user) {
        store.put(user.getId(), user);
        return user;
    }

    @Override
    public Optional<User> findById(Long id) {
        return Optional.ofNullable(store.get(id));
    }
}
```

Comparaison des doubles de test

Comparatif des doubles de test

Type	Retourne des valeurs	Vérifie les appels	Logique interne
Dummy	Non (null/vide)	Non	Non
Stub	Oui (prédéfini)	Non	Non
Spy	Oui (enregistre)	Optionnel	Optionnel
Mock	Oui (configurable)	Oui	Non
Fake	Oui	Non	Oui (simplifiée)

VS Mock vs Stub — Distinction fondamentale

Stub — Test d'état

```
// On vérifie l'état résultant
UserRepository stub = new InMemoryUserRepo();
UserService service = new UserService(stub);

service.inscrire(nouvelUser);

// Assertion sur l'état
assertEquals(1, stub.count());
```

Mock — Test de comportement

```
// On vérifie les interactions
UserRepository mock = mock(UserRepository.class);
UserService service = new UserService(mock);

service.inscrire(nouvelUser);

// Assertion sur le comportement
verify(mock).save(nouvelUser);
```


| 03 · Implémentation sans Bibliothèque

Comprendre les mécanismes

Pourquoi implémenter manuellement ?

- Comprendre **les principes fondamentaux** des doubles de test
- Pas toujours besoin d'une bibliothèque pour des cas simples
- Certains projets ont des **contraintes sur les dépendances**
- Mieux comprendre ce que Mockito fait "sous le capot"

"Avant d'utiliser un outil, comprends ce qu'il fait pour toi."

Stub manuel — Exemple complet

```
// Interface à stuber
public interface PrixService {
    double getPrixUnitaire(String produitId);
    boolean estDisponible(String produitId);
}

// Stub pour les tests
class StubPrixService implements PrixService {
    @Override
    public double getPrixUnitaire(String produitId) {
        return switch (produitId) {
            case "PROD001" -> 29.99;
            case "PROD002" -> 49.99;
            default -> 0.0;
        };
    }

    @Override
    public boolean estDisponible(String produitId) {
        return !produitId.equals("PROD999"); // tout sauf PROD999
    }
}
```

Spy manuel — Exemple complet

```
class SpyNotificationService implements NotificationService {
    // Enregistrement des appels
    private int nombreAppels = 0;
    private List<String> destinatairesNotifies = new ArrayList<>();
    private String dernierMessage;

    @Override
    public void notifier(String destinataire, String message) {
        nombreAppels++;
        destinatairesNotifies.add(destinataire);
        dernierMessage = message;
        // Optionnel : déléguer à l'implémentation réelle
    }

    // Méthodes d'inspection pour les tests
    public int getNombreAppels() { return nombreAppels; }
    public boolean aEteNotifie(String dest) {
        return destinatairesNotifies.contains(dest);
    }
    public String getDernierMessage() { return dernierMessage; }
}
```

Test avec un Spy manuel

```
@Test
void effectuerVirement_Reussi_NotifieDestinataire() {
    // Arrange
    SpyNotificationService spyNotif = new SpyNotificationService();
    VirementService service = new VirementService(
        new StubBanqueAPI(), spyNotif, new InMemoryVirementRepo()
    );
    Virement virement = new Virement("Alice", "Bob", 200.0);

    // Act
    service.effectuer(virement);

    // Assert – vérification du comportement
    assertEquals(1, spyNotif.getNombreAppels());
    assertTrue(spyNotif.aEteNotifie("Bob"));
    assertNotNull(spyNotif.getDernierMessage());
}
```



Mock manuel avec interface

```
// Mock qui vérifie lui-même les interactions
class MockEmailService implements EmailService {
    private boolean envoiAttendu = false;
    private String destinataireAttendu;
    private boolean envoiEffectue = false;

    public void attendreEnvoi(String destinataire) {
        envoiAttendu = true;
        destinataireAttendu = destinataire;
    }

    @Override
    public void envoyer(String destinataire, String contenu) {
        envoiEffectue = true;
        if (envoiAttendu && !destinataire.equals(destinataireAttendu)) {
            throw new AssertionError(
                "Email envoyé à " + destinataire +
                " mais attendu à " + destinataireAttendu
            );
        }
    }

    public void verifier() {
        if (envoiAttendu && !envoiEffectue)
            throw new AssertionError("Email non envoyé !");
    }
}
```



Limites des doubles manuels

Avantages

- Aucune dépendance externe
- Comportement parfaitement contrôlé
- Idéal pour comprendre les mécanismes

Inconvénients

- **Verbeux** : beaucoup de code à écrire et maintenir
- **Fragile** : doit être mis à jour si l'interface change
- **Limité** : difficile de gérer des comportements complexes
- **Duplication** : même logique répétée dans plusieurs tests

→ C'est pour ça que Mockito existe !

| 04 · Bibliothèques de Mocking

L'état de l'art

Les bibliothèques du marché

- **Mockito** ← Le plus populaire en Java
- **EasyMock** ← Pionnier, moins utilisé aujourd'hui
- **PowerMock** ← Mock de méthodes statiques/constructeurs
- **JMock** ← Orienté behavior-driven
- **WireMock** ← Mock de services HTTP/REST
- **MockServer** ← Mock de serveurs HTTP
- **Testcontainers** ← Vraies dépendances en Docker
- **Awaitility** ← Tests asynchrones



Comparaison des bibliothèques

Bibliothèque	Usage principal	Popularité	Particularité
Mockito	Mocking général	★★★★★	Simple, fluide
EasyMock	Mocking général	★★	Record/replay
PowerMock	Statiques, finaux	★★★	Extension Mockito
WireMock	HTTP REST	★★★★★	Serveur embarqué
Testcontainers	BDD, services	★★★★★	Docker intégré

Pourquoi choisir Mockito ?

- **Syntaxe fluide** et intuitive (`when().thenReturn()`)
- Intégration native avec **JUnit 5** via `@ExtendWith`
- **Largement adopté** : communauté massive, exemples partout
- Support de **Spring Boot Test** (`@MockBean`)
- **Active development** : régulièrement mis à jour
- Excellente **documentation** et stack overflow coverage

| 05 · Mockito — Fondamentaux

L'outil incontournable du TDD Java

Setup Mockito

```
<!-- Maven -->
<dependency>
  <groupId>org.mockito</groupId>
  <artifactId>mockito-junit-jupiter</artifactId>
  <version>5.5.0</version>
  <scope>test</scope>
</dependency>
```

```
// Gradle
testImplementation 'org.mockito:mockito-junit-jupiter:5.5.0'
```



Configuration avec @ExtendWith

```
import org.mockito.junit.jupiter.MockitoExtension;
import org.mockito.Mock;
import org.mockito.InjectMocks;

@ExtendWith(MockitoExtension.class) // ← Intégration JUnit 5
class VirementServiceTest {

    @Mock
    BanqueAPI banqueAPIMock; // ← Mockito crée le mock

    @Mock
    EmailService emailServiceMock; // ← Mockito crée le mock

    @InjectMocks
    VirementService virementService; // ← Mockito injecte les mocks
}
```

Créer un mock — 3 façons

```
// Méthode 1 – Statique (sans annotation)
import static org.mockito.Mockito.*;
userRepository repo = mock(UserRepository.class);

// Méthode 2 – Annotation @Mock (avec @ExtendWith)
@Mock
userRepository repo;

// Méthode 3 – MockitoAnnotations (sans extension JUnit)
@BeforeEach
void setUp() {
    MockitoAnnotations.openMocks(this);
}
```

Stubbing — when().thenReturn()

```
// Retourner une valeur simple
when(repo.findById(1L)).thenReturn(Optional.of(new User("Alice")));

// Retourner null
when(repo.findById(999L)).thenReturn(Optional.empty());

// Retourner des valeurs successives
when(service.getStatut())
    .thenReturn("EN_COURS")
    .thenReturn("TERMINÉ")
    .thenReturn("ARCHIVÉ");

// Retourner différentes valeurs selon l'argument
when(prix.getPrix(anyString())).thenReturn(10.0);
when(prix.getPrix("PREMIUM")).thenReturn(99.99);
```


! Stubbing — Lancer des exceptions

```
// Lancer une exception sur appel
when(banqueAPI.transferer(any()))
    .thenThrow(new BanqueException("Service indisponible"));

// Lancer une exception checked
when(banqueAPI.transferer(any()))
    .thenThrow(new IOException("Connexion perdue"));

// Comportement conditionnel
when(repo.findById(anyLong()))
    .thenReturn(Optional.empty())
    .thenThrow(new DatabaseException("Timeout"));
```

Stubbing — thenAnswer pour comportement dynamique

```
// thenAnswer – logique dynamique basée sur les arguments
when(repo.findByNom(anyString())).thenAnswer(invocation -> {
    String nom = invocation.getArgument(0);
    if (nom.startsWith("A")) return Optional.of(new User(nom));
    return Optional.empty();
});

// Simuler un délai (tests de timeout)
when(service.appelerAPI()).thenAnswer(invocation -> {
    Thread.sleep(5000); // simule latence réseau
    return "résultat";
});

// Déléguer à l'implémentation réelle
when(service.calculer(anyDouble())).thenCallRealMethod();
```

✓ Vérification — verify()

```
// Vérifier qu'une méthode a été appelée une fois
verify(emailService).envoyer("alice@email.com", anyString());

// Vérifier le nombre d'appels exact
verify(repo, times(3)).save(any());

// Vérifier qu'une méthode n'a JAMAIS été appelée
verify(emailService, never()).envoyer(anyString(), anyString());

// Vérifier au moins une fois
verify(repo, atLeastOnce()).save(any());

// Vérifier au plus N fois
verify(repo, atMost(2)).delete(anyLong());
```

Matchers d'arguments

```
// Matcher universel
when(service.traiter(any())).thenReturn(resultat);
when(service.traiter(any(Commande.class))).thenReturn(resultat);

// Matchers spécifiques
when(repo.findByAge(anyInt())).thenReturn(users);
when(service.rechercher(anyString())).thenReturn(resultats);
when(service.paginer(anyInt(), anyInt())).thenReturn(page);

// Valeurs exactes + matchers (RÈGLE : tout ou rien)
when(service.methode(eq("exact"), anyInt())).thenReturn(val);

// Matchers personnalisés
when(service.traiter(argThat(c -> c.getMontant() > 0))).thenReturn(ok);
```



ArgumentCaptor — Capturer les arguments

```
@Test
void inscrire_NouvelUtilisateur_SauvegardeAvecRoleParDefaut() {
    // Arrange
    ArgumentCaptor<User> userCaptor = ArgumentCaptor.forClass(User.class);

    // Act
    service.inscrire(new InscriptionRequest("Alice", "alice@email.com"));

    // Assert – capturer l'argument passé à save()
    verify(repo).save(userCaptor.capture());
    User userSauvegarde = userCaptor.getValue();

    assertEquals("Alice", userSauvegarde.getNom());
    assertEquals("alice@email.com", userSauvegarde.getEmail());
    assertEquals(Role.USER, userSauvegarde.getRole()); // rôle par défaut
}
```

ArgumentCaptor — Exemple avancé

```
@Test
void effectuerVirement_Reussi_EnvoieEmailAvecMontant() {
    // Arrange
    ArgumentCaptor<String> sujetCaptor = ArgumentCaptor.forClass(String.class);
    ArgumentCaptor<String> corpsCaptor = ArgumentCaptor.forClass(String.class);
    Virement virement = new Virement("Alice", "Bob", 500.0);

    // Act
    virementService.effectuer(virement);

    // Assert
    verify(emailService).envoyer(
        eq("Bob"),
        sujetCaptor.capture(),
        corpsCaptor.capture()
    );
    assertThat(sujetCaptor.getValue()).contains("virement");
    assertThat(corpsCaptor.getValue()).contains("500");
}
```



@Spy — Espionner les objets réels

```
// Spy sur un objet réel – utilise l'implémentation réelle par défaut
@Spy
List<String> listeReelle = new ArrayList<>();

// Spy via annotation
@Spy
CalculateurTVA calculateur; // ← implémentation réelle utilisée

@Test
void calculer_AppelleMethodeInterne() {
    // Stubbing partiel : surcharger une méthode spécifique
    doReturn(0.20).when(calculateur).getTaux("STANDARD");

    double resultat = calculateur.calculer(100.0, "STANDARD");

    verify(calculateur).getTaux("STANDARD"); // vérification du comportement
    assertEquals(20.0, resultat, 0.001);
}
```

⚡ doReturn vs thenReturn

```
// thenReturn – syntaxe standard (appelle la méthode réelle une fois)
when(service.methode()).thenReturn(valeur);

// doReturn – à utiliser avec les @Spy pour éviter l'appel réel
doReturn(valeur).when(spy).methode();

// doThrow – lancer exception sur méthode void
doThrow(new RuntimeException("erreur"))
    .when(service).methodeVoid();

// doNothing – ne rien faire (méthodes void)
doNothing().when(service).enregistrer(any());

// doAnswer – comportement dynamique sur void
doAnswer(invocation -> {
    System.out.println("Appelé avec : " + invocation.getArgument(0));
    return null;
}) when(service).notifier(anyString());
```


✓ Vérification de l'ordre des appels

```
@Test
void processus_EtapesOrdreCorrect_ReussitSequence() {
    // Act
    processus.executer(commande);

    // Assert – vérifier l'ordre des appels
    InOrder inOrder = inOrder(validationService, paiementService, emailService);

    inOrder.verify(validationService).valider(commande);
    inOrder.verify(paiementService).debiter(commande.getMontant());
    inOrder.verify(emailService).confirmer(commande.getClient());
}
```



END verifyNoMoreInteractions

```
@Test
void traiterCommande_Basique_SeulesInteractionsAttendues() {
    // Act
    service.traiter(commandeSimple);

    // Vérifier les interactions attendues
    verify(repo).save(any());
    verify(emailService).envoyer(anyString(), anyString());

    // Vérifier qu'il n'y a eu AUCUNE autre interaction
    verifyNoMoreInteractions(repo, emailService);
    // Échoue si une autre méthode a été appelée sur ces mocks
}
```

| 06 · Mockito — Fonctionnalités Avancées

Aller plus loin

@InjectMocks — Injection automatique

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    PaymentGateway paymentGateway;

    @Mock
    InventoryService inventoryService;

    @Mock
    NotificationService notificationService;

    @InjectMocks
    OrderService orderService;
    // Mockito injecte les 3 mocks ci-dessus dans OrderService
    // Par constructeur, setter ou champ (dans cet ordre de priorité)
}
```

⚙️ Mockito Settings — Configuration avancée

```
// Activer la vérification stricte des stubs (recommandé)
@ExtendWith(MockitoExtension.class)
// Par défaut avec mockito-junit-jupiter : STRICT_STUBS activé

// STRICT_STUBS détecte :
// - Stubs déclarés mais jamais appelés (stubs inutiles)
// - Stubs appelés mais avec de mauvais arguments
// - Arguments correspondant à un stub mais non utilisés

// Désactiver pour un test particulier si nécessaire
@MockitoSettings(strictness = Strictness.LENIENT)
class MonTestAvecStubsOptionnels { }
```



Mock de classes concrètes

```
// Mockito peut mocker des classes concrètes (pas seulement des interfaces)
// Attention : méthodes final ne peuvent pas être mockées par défaut

// Mocker une classe concrète
ArrayList<String> mockList = mock(ArrayList.class);
when(mockList.size()).thenReturn(42);

// Pour mocker des méthodes final → utiliser MockMaker inline
// mockito-extensions/org.mockito.plugins.MockMaker
// org.mockito.internal.creation.bytebuddy.InlineByteBuddyMockMaker
```

Mockito avec Spring Boot

```
// @MockBean – Mock g  r   par le contexte Spring
@SpringBootTest
class UserControllerIntegrationTest {

    @Autowired
    MockMvc mockMvc;

    @MockBean
    UserService userService; //    remplace le vrai bean Spring

    @Test
    void getUser_UserExiste_Retourne200() throws Exception {
        when(userService.findById(1L)).thenReturn(new User("Alice"));

        mockMvc.perform(get("/api/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.nom").value("Alice"));
    }
}
```

Exemple complet — VirementService

```
@ExtendWith(MockitoExtension.class)
class VirementServiceTest {

    @Mock BanqueAPI banqueAPI;
    @Mock EmailService emailService;
    @Mock VirementRepository repository;
    @InjectMocks VirementService virementService;

    @Test
    void effectuerVirement_Reussi_SauvegardeEtNotifie() {
        Virement virement = new Virement("Alice", "Bob", 500.0);
        when(banqueAPI.transférer(virement)).thenReturn(true);

        virementService.effectuer(virement);

        verify(repository).save(virement);
        verify(emailService).notifier("Bob", any());
    }

    @Test
    void effectuerVirement_EchecBanque_LeveException() {
        Virement virement = new Virement("Alice", "Bob", 500.0);
        when(banqueAPI.transférer(virement)).thenReturn(false);

        assertThrows(VirementException.class,
            () -> virementService.effectuer(virement));
        verify(repository, never()).save(any());
    }
}
```


TP 3 — Mockito Fondamentaux

Objectif

Maîtriser la création et l'utilisation de mocks Mockito

Scénario

Implémenter un **système de réservation de billets** :

- Réserver un billet (vérifier disponibilité, débiter, confirmer)
- Annuler une réservation (rembourser, notifier)
- Lister les réservations d'un utilisateur

TP 3 — Tests à implémenter

```
// Service à tester
class ReservationService {
    ReservationService(DisponibiliteService dispo,
                       PaiementService paiement,
                       NotificationService notif,
                       ReservationRepository repo) {}
    Reservation reserver(String userId, String billetId) {}
    void annuler(String reservationId) {}
}

// Tests attendus
void reserver_BilletDisponible_ConfirmeEtNotifie()
void reserver_BilletIndisponible_LeveException()
void reserver_EchecPaiement_NeSauvegardeRien()
void annuler_ReservationExistante_RembourseEtNotifie()
void annuler_ReservationInexistante_LeveException()
```

| 07 · Fixtures et Organisation des Tests

Structure et réutilisabilité



Qu'est-ce qu'une Fixture ?

*Une **fixture** est l'ensemble des données et objets nécessaires pour qu'un test puisse s'exécuter dans un état connu et contrôlé.*

Types de fixtures

- **Données de test** : objets, valeurs, collections pré-configurés
- **Infrastructure** : base de données, serveurs embarqués
- **Contexte** : état du système avant l'exécution du test



Cycle de vie des fixtures

```
class MonServiceTest {  
  
    // ← Exécuté UNE FOIS avant tous les tests de la classe  
    @BeforeAll  
    static void initialiserInfrastructure() {  
        // ex: démarrer un serveur embarqué, une BDD H2  
    }  
  
    // ← Exécuté avant CHAQUE test  
    @BeforeEach  
    void preparerContexte() {  
        // ex: créer les objets de test, réinitialiser l'état  
    }  
  
    // ← Exécuté après CHAQUE test  
    @AfterEach  
    void nettoyerContexte() {  
        // ex: supprimer les données créées pendant le test  
    }  
  
    // ← Exécuté UNE FOIS après tous les tests  
    @AfterAll  
    static void arreterInfrastructure() {  
        // ex: arrêter les serveurs embarqués  
    }  
}
```

Fixtures partagées avec @BeforeEach

```
class CommandeServiceTest {
    private CommandeService service;
    private User userAlice;
    private Produit produitA;

    @BeforeEach
    void setUp() {
        // Fixtures recréées à chaque test → isolation garantie
        userAlice = new User(1L, "Alice", "alice@email.com");
        produitA = new Produit("P001", "Widget", 29.99);

        UserRepository mockRepo = mock(UserRepository.class);
        ProduitRepository mockProduit = mock(ProduitRepository.class);

        when(mockRepo.findById(1L)).thenReturn(Optional.of(userAlice));
        when(mockProduit.findById("P001")).thenReturn(Optional.of(produitA));

        service = new CommandeService(mockRepo, mockProduit);
    }
}
```

Test Data Builders — Pattern Builder

```
// Builder pour créer des objets de test expressifs
class UserBuilder {
    private Long id = 1L;
    private String nom = "DefaultUser";
    private String email = "default@test.com";
    private Role role = Role.USER;
    private boolean actif = true;

    public UserBuilder avecId(Long id) { this.id = id; return this; }
    public UserBuilder avecNom(String nom) { this.nom = nom; return this; }
    public UserBuilder avecEmail(String email) { this.email = email; return this; }
    public UserBuilder admin() { this.role = Role.ADMIN; return this; }
    public UserBuilder inactif() { this.actif = false; return this; }
    public User build() { return new User(id, nom, email, role, actif); }

    public static UserBuilder unUser() { return new UserBuilder(); }
}
```

Utilisation du Builder dans les tests

```
// ✗ Sans builder – verbeux et peu lisible
User user = new User(null, "Alice", "alice@email.com", Role.USER, true);

// ✔ Avec builder – expressif et lisible
User alice = unUser().avecNom("Alice").avecEmail("alice@email.com").build();
User admin = unUser().avecNom("Admin").admin().build();
User userInactif = unUser().inactif().build();

// Contexte du test clairement exprimé
@Test
void accesDashboard_AdminActif_AutoriseAcces() {
    User admin = unUser().admin().build();
    assertTrue(securiteService.autoriserAcces(admin, "/admin/dashboard"));
}
```




Object Mother — Alternative au Builder

```
// Object Mother : classe utilitaire avec des méthodes de fabrication
class Mothers {
    // Utilisateurs pré-configurés
    public static User unNouvelUtilisateur() {
        return new User(null, "Nouveau", "nouveau@test.com", Role.USER, true);
    }

    public static User unUtilisateurActif() {
        return new User(1L, "Alice", "alice@test.com", Role.USER, true);
    }

    public static User unAdministrateur() {
        return new User(99L, "Admin", "admin@test.com", Role.ADMIN, true);
    }

    // Commandes pré-configurées
    public static Commande uneCommandeSimple() {
        return new Commande(Mothers.unUtilisateurActif(), List.of());
    }
}
```

Organiser les tests — Conventions

Structure recommandée

```
class CommandeServiceTest {  
    // Groupe 1 : tests nominaux (happy path)  
    @Nested  
    @DisplayName("Cas nominaux")  
    class CasNominauxTest { }  
  
    // Groupe 2 : tests de validation  
    @Nested  
    @DisplayName("Validation des données")  
    class ValidationTest { }  
  
    // Groupe 3 : gestion des erreurs  
    @Nested  
    @DisplayName("Gestion des erreurs")  
    class GestionErreursTest { }  
}
```

| 08 · Qualité du Code de Test

Des tests qui durent dans le temps

Qualités d'un bon test

Les 4 critères RITS

- **R**eadable — lisible comme une spécification
- **I**solated — indépendant de l'environnement et des autres tests
- **T**horough — couvre les cas nominaux ET les cas limites
- **S**peedy — rapide à exécuter (< 100ms pour un test unitaire)

Lisibilité — Tests comme documentation

```
// ✗ Test peu lisible
@Test
void t1() {
    CB c = new CB(500);
    c.dep(100);
    assertEquals(600, c.getSol(), 0.01);
}

// ✓ Test lisible – spécification exécutable
@Test
@DisplayName("Un dépôt de 100€ sur un compte de 500€ donne 600€")
void deposer_100Euros_SurCompteDe500Euros_DonneSolDe600Euros() {
    // Arrange
    CompteBancaire compte = new CompteBancaire(500.0);

    // Act
    compte.deposer(100.0);

    // Assert
    assertEquals(600.0, compte.getSolde(), 0.01,
        "Le solde devrait être de 600€ après un dépôt de 100€");
}
```

Les Messages d'assertion

```
// ❌ Pas de message → difficile à déboguer
assertEquals(700.0, compte.getSolde());

// ✅ Message explicite → diagnostic immédiat
assertEquals(700.0, compte.getSolde(), 0.001,
    "Après dépôt de 200€ sur un solde de 500€, le solde devrait être 700€");

// ✅ AssertJ – message automatiquement lisible
assertThat(compte.getSolde())
    .as("Solde après dépôt de 200€")
    .isEqualTo(700.0);
```

Éviter la duplication dans les tests

```
// ✗ Code dupliqué dans chaque test
@Test
void test1() {
    VirementService service = new VirementService(
        mock(BanqueAPI.class), mock(EmailService.class), mock(Repo.class)
    );
    // test...
}

@Test
void test2() {
    VirementService service = new VirementService(
        mock(BanqueAPI.class), mock(EmailService.class), mock(Repo.class)
    );
    // même setup...
}

// ✔ Extraction dans @BeforeEach ou méthode helper
private VirementService creerServiceAvecMocks() {
    return new VirementService(
        mock(BanqueAPI.class), mock(EmailService.class), mock(Repo.class)
    );
}
```

⚠ Code smells dans les tests

À détecter et corriger

- **Test trop long** : > 20 lignes → test trop de choses
- **Arrange trop complexe** : > 10 lignes → mauvaise conception
- **Tests couplés** : l'ordre d'exécution importe → isolation manquante
- **Vérification de l'implémentation** : `verify(mock, times(3))` → fragile
- **Logique dans les tests** : `if`, `for`, `switch` → tests inconditionnels
- **Données magiques** : `assertEquals(42, résultat)` → nommer les constantes

Logique dans les tests

```
// ❌ Logique dans le test – fragile et difficile à comprendre
@Test
void testMultiplesUtilisateurs() {
    List<User> users = Arrays.asList(user1, user2, user3);
    for (User user : users) {
        if (user.isActif()) {
            assertNotNull(service.login(user));
        } else {
            assertThrows(Exception.class, () -> service.login(user));
        }
    }
}

// ✅ Tests séparés et inconditionnels
@ParameterizedTest
@MethodSource("utilisateursActifs")
void login_UtilisateurActif_ReussitConnexion(User user) { ... }

@ParameterizedTest
@MethodSource("utilisateursInactifs")
void login_UtilisateurInactif_LeveException(User user) { ... }
```



Encapsulation dans les tests

```
// ❌ Le test accède aux internals de l'objet
@Test
void testInternals() {
    CompteBancaire compte = new CompteBancaire(500.0);
    compte.deposer(100.0);

    // Accès au champ privé via réflexion – mauvaise pratique
    Field soldeField = CompteBancaire.class.getDeclaredField("solde");
    soldeField.setAccessible(true);
    assertEquals(600.0, soldeField.get(compte));
}

// ✅ Tester uniquement le comportement observable
@Test
void deposer_100Euros_SoldeAugmenteDe100() {
    CompteBancaire compte = new CompteBancaire(500.0);
    compte.deposer(100.0);
    assertEquals(600.0, compte.getSolde()); // méthode publique
}
```

| 09 · Tests basés sur la Responsabilité

Tester le bon niveau d'abstraction

Tests basés sur la responsabilité

Principe

Tester ce que la classe/méthode **est responsable de faire**, pas comment elle le fait.

Questions à se poser

- Quelle est la **responsabilité** de cette méthode ?
- Quel **comportement** doit-elle exposer ?
- Quelles **garanties** doit-elle offrir à ses clients ?

Responsabilité vs Implémentation

Responsabilité

"CommandeService est responsable de créer une commande valide, de débiter le paiement et d'envoyer une confirmation."

Ce qu'on teste

```
// ✅ Tester la responsabilité
void creerCommande_Valide_EnvoieConfirmationEmail()
void creerCommande_Valide_DebiteLePaiement()
void creerCommande_PaiementEchoue_NeCreePasCommande()

// ❌ Tester l'implémentation
void creerCommande_AppellePaymentServiceDebitTroisFois()
void creerCommande_UtiliseStrategyPatternPourEmail()
```

Contrat d'une méthode

Chaque méthode a un contrat implicite

- **Préconditions** : ce qu'elle attend en entrée
- **Postconditions** : ce qu'elle garantit en sortie
- **Invariants** : ce qui ne change pas

Tester le contrat

```
// Tester les préconditions (cas invalides)
void creerCommande_UserNull_LeveIllegalArgumentException()
void creerCommande_ArticlesVides_LeveIllegalStateException()

// Tester les postconditions (cas valides)
void creerCommande_DataValide_RetourneCommandeAvecId()
void creerCommande_DataValide_StatutInitialEnAttente()
```

🧩 Tests basés sur l'implémentation

Problème

```
// Refactoring : on passe de Strategy Pattern à if/else
// Le comportement est identique mais le test échoue !

// ❌ Fragile – couplé à l'implémentation
@Test
void calculer_AppelleStrategyCalcul() {
    verify(strategyMock).calculer(anyDouble()); // Échoue après refactoring
}

// ✅ Robuste – couplé au comportement
@Test
void calculer_MontantValide_RetourneValeurCorrecte() {
    assertEquals(20.0, calculateur.calculer(100.0, 0.20), 0.001); // Résiste au refactoring
}
```

| 10 · Styles de TDD

Classique vs London

Deux Écoles du TDD

École Classique (Detroit School)

- Fondée par Kent Beck et Ward Cunningham
- Préfère les **vrais objets** et les **fakes** aux mocks
- Teste l'**état** plutôt que le comportement
- Design émerge de l'**intérieur vers l'extérieur** (bottom-up)

École London (Mockist School)

- Popularisée par Steve Freeman et Nat Pryce
- Utilise largement les **mocks** pour isoler
- Teste le **comportement** et les interactions
- Design émerge de l'**extérieur vers l'intérieur** (top-down)



Style Classique — Caractéristiques

```
// Classique : on utilise de vraies implémentations quand possible
@Test
void traiterCommande_ValideAvecFakeRepo_SauvegardeCommande() {
    // Fake (implémentation en mémoire)
    InMemoryCommandeRepo repo = new InMemoryCommandeRepo();
    CommandeService service = new CommandeService(repo, new StubPaielement());

    service.traiter(commande);

    // Assert sur l'état
    assertEquals(1, repo.count());
    assertEquals(commande, repo.findAll().get(0));
}
```

Avantages

- Tests plus proches du comportement réel
- Moins sensibles aux changements d'implémentation
- Pas de problème de sur-spécification



Style London — Caractéristiques

```
// London : tout est mocké pour isoler parfaitement
@ExtendWith(MockitoExtension.class)
class CommandeServiceLondonTest {

    @Mock CommandeRepository repo;
    @Mock PaiementService paiement;
    @InjectMocks CommandeService service;

    @Test
    void traiterCommande_Valide_DebitePuisSauvegarde() {
        service.traiter(commande);

        // Assert sur le comportement
        InOrder order = inOrder(paiement, repo);
        order.verify(paiement).debiter(commande.getMontant());
        order.verify(repo).save(commande);
    }
}
```



Classique vs London — Comparaison

Critère	Classique	London
Isolation	Par module	Par classe
Dépendances	Vraies / Fakes	Mocks
Ce qu'on teste	État	Comportement
Direction	Bottom-up	Top-down
Fragilité	Plus stable	Plus fragile
Feedback	Plus lent	Plus rapide
Design	Émerge	Guidé

Approche pragmatique

Recommandation

Il n'y a pas de bonne réponse unique — choisir selon le contexte.

Règles pragmatiques

- **Logique métier pure** → Style Classique (état, vraies implémentations)
- **Interactions avec services externes** → Mocks (BDD, API, emails)
- **Architecture hexagonale** → Fakes pour les adaptateurs
- **Tests d'intégration** → Testcontainers ou H2

Outside-in TDD (London style)

Processus

1. Écrire un test d'acceptation qui échoue (Failing Acceptance Test)
2. Écrire un test unitaire pour la première classe à implémenter
3. Implémenter la classe (TDD inner loop)
4. Répéter 2-3 jusqu'à ce que le test d'acceptation passe
5. Refactorer

Avantage

- Garantit que tout le code écrit **sert un besoin réel**
- Architecture guidée par les **cas d'utilisation**

TP 4 — Mockito Avancé

Objectif

Maîtriser ArgumentCaptor, vérification d'ordre, stubs conditionnels

Scénario

Implémenter un **système de notifications multi-canaux** :

- NotificationService envoie par Email, SMS, et Push
- Selon le profil utilisateur et la priorité du message
- Retry automatique en cas d'échec

TP 4 — Tests à écrire

```
// Tests avec ArgumentCaptor
void envoyer_EmailUrgent_InclueMentionUrgentDansSujet()
void envoyer_SMSLong_TronqueAu160Caracteres()

// Tests d'ordre
void envoyerUrgent_LivreParEmailPuisSMS_DansOrdre()

// Tests de retry
void envoyer_EchecEmail_RessaieDeuxFois()
void envoyer_TroisEchecs_LeveNotificationException()

// Tests de comportement conditionnel
void envoyer_UtilisateurSansTelephone_IgnoreSMS()
void envoyer_NotifPush_SeulementSiAppInstalle()
```


| 11 • Techniques d'Écriture Avancées

Patterns et stratégies

Tests basés sur les propriétés

Property-Based Testing

- Au lieu de cas spécifiques → tester des **propriétés** générales
- L'outil génère des données aléatoires qui satisfont des conditions

```
// Test classique
void additionner_DeuxPositifs_RetournePositif() {
    assertTrue(calc.additionner(5, 3) > 0);
}

// Property-Based avec jqwik
@property
void additionner_DeuxPositifs_RetourneToujoursPositif(
    @forall @positive int a,
    @forall @positive int b) {
    assertTrue(calc.additionner(a, b) > 0);
}
```

▲ Tests basés sur la partition

Équivalence Partitioning

Diviser les entrées en **classes d'équivalence** — si un cas représentatif fonctionne, tous les cas de la classe fonctionnent.

Entrée : âge pour accès site adulte

Partitions :

- Valeurs négatives $[-\infty, -1]$ → `IllegalArgumentException`
- Mineurs $[0, 17]$ → Accès refusé
- Adultes $[18, 120]$ → Accès autorisé
- Hors limites $[121, \infty]$ → `IllegalArgumentException`

Analyse des valeurs limites

Boundary Value Analysis

Tester les **valeurs aux frontières** : elles concentrent souvent les bugs

```
// Pour age >= 18 → accès adulte
@ParameterizedTest
@CsvSource({
    "17, false",    // juste en-dessous de la limite
    "18, true",     // exactement à la limite
    "19, true",     // juste au-dessus
    "0, false",     // minimum absolu
    "120, true",    // maximum raisonnable
    "-1, false"     // hors limites inférieur
})
void verifierAge_ValeurLimite_AccesCorrect(int age, boolean accesAttendu) {
    assertEquals(accesAttendu, service.autoriserAcces(age));
}
```



Mutation Testing

Principe

Introduire **délibérément des bugs** (mutations) dans le code et vérifier que les tests les **détectent**.

```
// Code original
public boolean estMajeur(int age) {
    return age >= 18;
}

// Mutations générées automatiquement
return age > 18;    // ← mutation: >= devient >
return age <= 18;   // ← mutation: >= devient <=
return true;       // ← retourne toujours true
return false;      // ← retourne toujours false
```

Outil Java : *PIT* (pitest.org) — intégrable avec Maven

⚙️ PIT Mutation Testing — Setup

```
<plugin>
  <groupId>org.pitest</groupId>
  <artifactId>pitest-maven</artifactId>
  <version>1.14.0</version>
  <dependencies>
    <dependency>
      <groupId>org.pitest</groupId>
      <artifactId>pitest-junit5-plugin</artifactId>
      <version>1.2.0</version>
    </dependency>
  </dependencies>
  <configuration>
    <targetClasses>com.example.*</targetClasses>
    <targetTests>com.example.*Test</targetTests>
  </configuration>
</plugin>
```

```
mvn org.pitest:pitest-maven:mutationCoverage
```



Tests de performance simples

```
// JUnit 5 – timeout simple
@Test
@Timeout(value = 200, unit = TimeUnit.MILLISECONDS)
void rechercher_1000Produits_SousSeuilPerf() {
    List<Produit> produits = service.rechercher("widget");
    assertThat(produits).isNotEmpty();
}

// Test de performance avec JMH (Java Microbenchmark Harness)
@Benchmark
@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.MICROSECONDS)
public Resultat benchmarkRecherche(Etat etat) {
    return etat.service.rechercher("widget");
}
```

Tests de snapshot

Principe

Comparer la sortie d'une méthode à un **résultat de référence enregistré**.

```
// Exemple avec ApprovalTests
@Test
void genererRapport_DonneesComplettes_CorrespondAuSnapshot() {
    Rapport rapport = service.generer(donnees);
    String json = objectMapper.writeValueAsString(rapport);

    // Compare avec le fichier référence (créé au premier run)
    Approvals.verify(json);
}

// Fichier rapport.approved.json généré automatiquement
// et versionné dans le dépôt Git
```


TP 5 — Techniques avancées

Objectif

Améliorer la qualité des tests écrits et appliquer les bonnes pratiques

Exercice 1 — Refactoring de tests

Réécrire des tests existants (fournis par le formateur) :

- Extraire les builders et Object Mothers
- Appliquer les bonnes pratiques de nommage
- Séparer les préoccupations avec `@Nested`

TP 5 — Suite

Exercice 2 — Analyse des valeurs limites

Implémenter des tests complets pour un **validateur de formulaire** :

- Champ email : format, longueur
- Champ âge : bornes, type
- Champ mot de passe : complexité, longueur

Exercice 3 — Partition des entrées

Identifier les classes d'équivalence et écrire les tests paramétrés correspondants

| 12 · Synthèse du Jour 2

Bilan et perspectives

Bilan — Jour 2 (1/2)

Doubles de test

- Les **5 types** : Dummy, Stub, Fake, Spy, Mock
- Différence fondamentale **Mock vs Stub** (comportement vs état)
- Implémentation **manuelle** pour comprendre les mécanismes
- Bibliothèques disponibles dans l'écosystème Java

Mockito

- Setup avec `@ExtendWith(MockitoExtension.class)` et `@Mock`
- Stubbing : `when().thenReturn()`, `thenThrow()`, `thenAnswer()`
- Vérification : `verify()`, `times()`, `never()`, `inOrder()`
- **ArgumentCaptor** pour inspecter les arguments passés

Bilan — Jour 2 (2/2)

Qualité des tests

- **Fixtures** : `@BeforeEach`, Test Data Builder, Object Mother
- **Anti-patterns** : logique dans les tests, couplage à l'implémentation
- Tests basés sur la **responsabilité** (pas l'implémentation)
- **Lisibilité** comme documentation exécutable

Styles de TDD

- **Classique** (Detroit) : état, fakes, bottom-up
- **London** (Mockist) : comportement, mocks, top-down
- Approche **pragmatique** selon le contexte



Programme — Jour 3

Ce qui nous attend demain

- **Tests de code hérité** : qu'est-ce que le legacy code ?
- **Cycle d'évolution** du code hérité
- **FitNesse** : tests fonctionnels et collaboration MOA/MOE
- **Outils d'intégration continue** : Jenkins, GitHub Actions
- **Outils de couverture** : JaCoCo, SonarQube
- **Architecture matérielle** des tests
- **Travaux pratiques** : mise en œuvre outillage complet

Ressources Jour 2

Livres

- *Growing Object-Oriented Software Guided by Tests* — Freeman & Pryce
- *xUnit Test Patterns* — Gerard Meszaros
- *Effective Java Testing* — Jonathan Raskin

Documentation

- **Mockito** : javadoc.io/doc/org.mockito/mockito-core
- **AssertJ** : assertj.github.io/assertj-core/
- **PIT Mutation** : pitest.org

Exercices du soir

Pour consolider les acquis

1. Implémenter un **service de panier e-commerce** en TDD style London
2. Créer des **Test Data Builders** pour tous les objets du domaine
3. Refactorer les tests TP 3 en appliquant les bonnes pratiques
4. Analyser la couverture de mutation avec PIT (si installé)

Objectif : 30-45 minutes de pratique autonome

QCM Jour 2 — Vérification

Question 1

Quelle est la différence principale entre un Mock et un Stub ?

- A) Les mocks sont plus rapides
- B) **Les mocks vérifient le comportement, les stubs fournissent des données** 
- C) Les stubs sont plus précis
- D) Il n'y a pas de différence

QCM Jour 2 — Suite

Question 2

Quelle méthode Mockito permet de capturer un argument passé à un mock ?

- A) `ArgumentSpy`
- B) `ArgumentMatcher`
- C) `ArgumentCaptor` ✓
- D) `ArgumentVerifier`

Question 3

Dans le style TDD London, quelle est l'approche de développement ?

- A) Bottom-up (de l'intérieur vers l'extérieur)
- B) **Top-down (de l'extérieur vers l'intérieur)** ✓
- C) Au hasard
- D) Uniquement par tests d'intégration

Question 4

Qu'est-ce qu'un Fake en tests unitaires ?

- A) Un objet qui ne fait rien
- B) Un objet qui lève toujours une exception
- C) **Une implémentation fonctionnelle mais simplifiée** ✓
- D) Un objet qui vérifie les appels

Question 5

Quand utiliser `doReturn().when()` plutôt que `when().thenReturn()` ?

- A) Jamais, ils sont équivalents
- B) **Avec les `@Spy` pour éviter d'appeler la méthode réelle** ✓
- C) Uniquement pour les méthodes void
- D) Quand l'argument est null



Fin du Jour 2

Excellente journée sur les Mocks et Mockito !

Rendez-vous demain pour le Code Hérité et les Outils CI



Pause & Questions

N'hésitez pas à poser vos questions !

Formation TDD Java · Jour 2 · © 2026